Do we need more bikes?
Project in Machine Learning

Subash K C
subash.k-c.9608@student.uu.se

Abstract

This project aims to predict whether the District Department of Transportation in Washington D.C. should increase the number of bikes available in the public bicycle-sharing system at a particular hour based on various temporal and meteorological factors such as date and time, weather conditions, temperature, and other relevant features, as a binary classification task. The dataset provided includes features such as temperature, humidity, precipitation, time of day, and holidays, which are analyzed to understand patterns in bike demand. The training set comprises 1600 instances of hourly bike rental features. Several classification models, such as Logistic Regression, K-Nearest Neighbors, and Random Forest, were explored. After evaluating the models using performance metrics such as accuracy, precision, recall, and F1-score, Random Forest emerged as the best model, providing the highest F1-score and accuracy. The final model will be used to make predictions on future, a given test set of 400 instances. "Number of group members: 1".

1 Introduction

Capital Bikeshare is a public bicycle system in Washington, D.C., serving thousands of residents and visitors daily. The problem that aries is that there are certain occasions, due to various reasons, there are not as many bikes available as there are demands. In the long term, this will lead to increased car use, contributing to higher CO2 emissions. To address this issue, the District Department of Transportation wants to identify periods when increasing the availability of bikes is necessary.

This requires an understanding of the factors that influence bike demand, such as time of day, weather conditions, holidays, and seasonal trends. In this project, we leverage machine learning techniques such as logistic regression, K-nearest neighbors, and random forest models to predict whether the number of bikes available at a given hour should be increased, based on these factors.

The dataset provided for this project includes various temporal and meteorological data, such as the hour of the day, day of the week, temperature, precipitation, and wind speed. The goal is to apply classification algorithms to predict whether there will be a high or low demand for bikes at a specific hour, thus helping the District Department of Transportation optimize the distribution of bikes and improve the efficiency of the bike-sharing program.

In this report, we explore the logistic regression, K-nearest neighbors, and random forest models, evaluate their performance, and select the best-performing model to be deployed to predict bike demand and to the given test set of 400 instances. The project aims to contribute to more sustainable urban transportation systems by ensuring that bike availability aligns with user demand.

2 Data Analysis

In this section, we analyze the relationships between the features in the dataset and the target variable, `increase_stock`, which indicates whether the number of bikes needs to be increased at a given time. The numerical features in the dataset include `temperature` (`temp`),

`Dew Point` (`dew`), `humidity` (`humidity`), `wind speed` (`windspeed`), `precipitation` (`precip`), `snow depth` (`snowdepth`), `visibility`(`visibility`), and Cloud `Cover` (`cloudcover`), while the categorical features are the column of `hour_of_day`, `day_of_week` , month, Holiday, weekday, summertime, and `increase_stock`.

2.1 Categorical Features vs. Bike Demand

Based from the analysis of Figure 1(1), several trends regarding bike demand can be identified.

- **Hours of the Day:** Bike demand is highest during midday to evening hours (12:00–19:00), which corresponds to peak commuting times. Conversely, during the late night (00:00–5:00), demand is minimal.
- **Months and Seasonal Trends:** Demand is generally higher during warmer months, such as in summertime, and lower during colder month. This is consistent with the bike demand trends observed during summertime versus non-summertime.
- **Weekdays vs. Weekends:** There is not a significant difference in bike demand between weekdays and weekends, although demand tends to be slightly higher on weekends. This is evident from the “bike demand vs. day of the week" analysis from 1, which shows that while weekday demand is high, weekends also see notable usage.
- **Holidays vs. Non-Holidays:** The most striking trend is that bike demand is considerably higher during non-holiday periods. On holidays, bike demand tends to drop. This pattern is clearly reflected in the “holiday vs. no holiday" comparison in 1, where demand is significantly higher when there are no holidays.

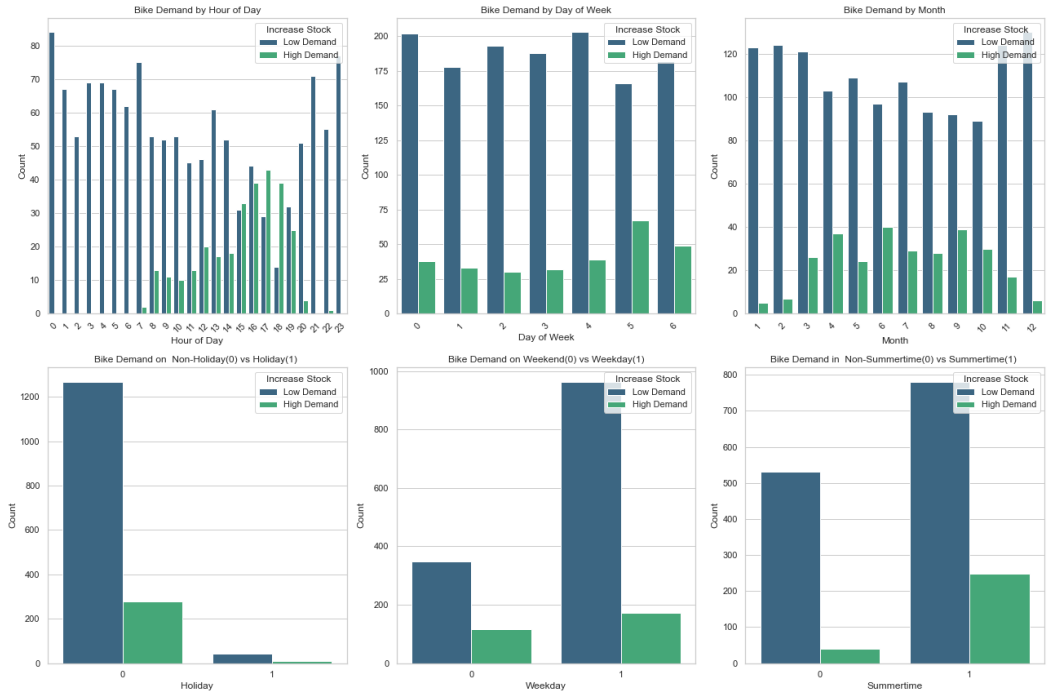

Figure 1: The distribution of bike demand across different categorical features .

2.2 Numerical Features vs. Bike Demand

For the numerical features, we divided the data into intervals and performed Chi-square tests to visualize the relationship between the features and bike demand. The results are presented in Figure 2 (2), where the plot illustrates how different weather conditions correspond to varying levels of bike

demand. This approach helps in understanding the impact of numerical features(wheather) on bike demand more clearly.

Based on the analysis and the visualizations in Figure 2(2), we can observe how various weather features impact bike demand. For example, Higher temperatures tend to correlate with increased bike demand, with higher proportions of high demand seen in warmer intervals (20-30°C and 30-40°C), while higher precipitation and snow significantly reduce it. Moderate humidity and low wind speeds show higher demand proportions, whereas extreme values in these features reduce demand. Cloud cover exhibits a minimal relationship with bike demand.

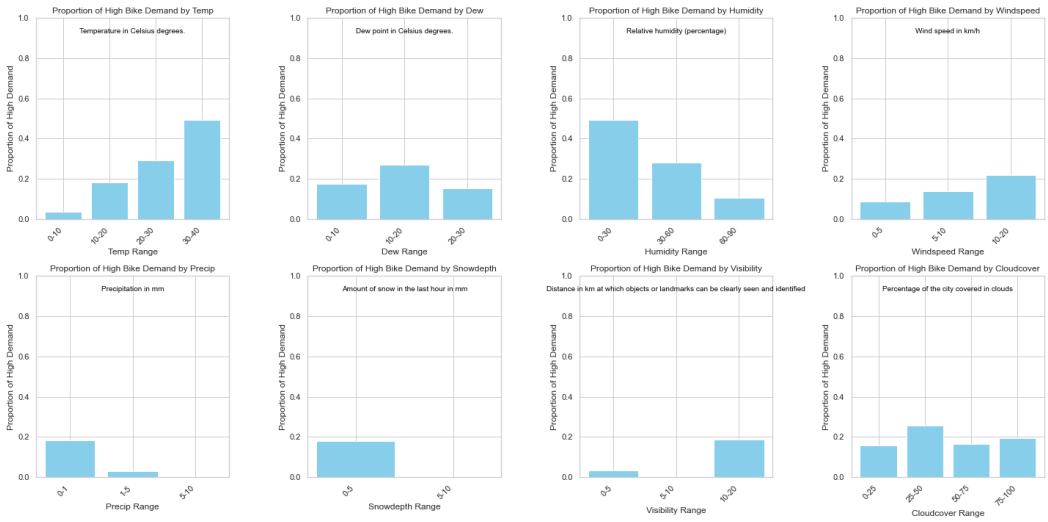

Figure 2: The distribution of bike demand across different numerical features .

3 Model Development

In this section, we describe the mathematical concepts behind the models considered, how they were applied to the data, and the evaluation of each method. Finally, we present our model selection process and justify our choice for deployment.

3.1 Mathematical Description of Each Method

We considered three different machine learning methods for predicting bike demand: Logistic Regression, K-Nearest Neighbors (KNN), and Random Forest. Below are the mathematical concepts behind each method.

3.1.1 Logistic Regression

Logistic Regression is a linear model primarily used for binary classification. It estimates the probability that a given input belongs to a particular class by applying the logistic function (also known as the sigmoid function), which outputs values between 0 and 1. The model is defined as follows:

$$P(y = 1|X) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_n X_n)}}$$

Where:

- $P(y = 1|X)$ is the probability that bike demand should be increased (target variable).
- $X_1, X_2, \ldots, X_n$ are the input features (e.g., temperature, hour of day).
- $\beta_0, \beta_1, \ldots, \beta_n$ are the coefficients learned during training.

The model's coefficients are estimated by minimizing the binary cross-entropy loss function. The binary cross-entropy loss function measures the difference between predicted probabilities ($\hat{y}$) and true binary labels (y):

$$\text{Loss} = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)]$$

Where:

- N is the number of samples.
- $y_i \in \{0, 1\}$ is the true label.
- $\hat{y}_i$ is the predicted probability for $y_i = 1$.

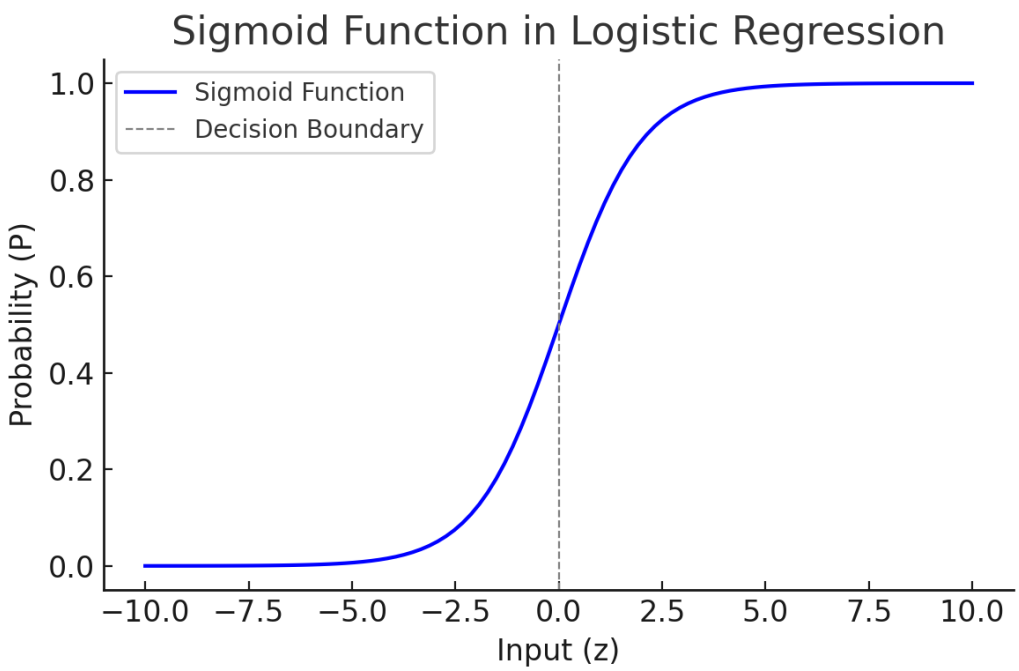

Figure 3: Graph of a logistic regression curve.

As illustrated in the figure (3) above, the sigmoid function transforms the input values into probabilities that lie between 0 and 1. This property makes Logistic Regression suitable for classification tasks where the goal is to predict a binary outcome.

Logistic Regression is particularly effective when there is a linear relationship between the input features and the log-odds of the target variable. If the relationship between the input features and the target is non-linear, Logistic Regression may not be able to capture the underlying patterns in the data effectively. The model will still try to fit a linear decision boundary (hyperplane) to separate the classes, which may result in poor performance, especially in cases where the data is more complex or the decision boundary is highly non-linear. Logistic Regression is computationally efficient and interpretable, making it a solid baseline model for classification tasks.

3.1.2 K-Nearest Neighbors (KNN)

K-Nearest Neighbors (KNN) is a non-parametric, instance-based learning algorithm that can be applied to both classification and regression tasks. Its core idea is to make predictions based on the k-nearest neighbors of a given test instance in the feature space. The model works by representing

each instance as a point in an n-dimensional feature space and calculating the distance between instances using a chosen metric, most commonly the Euclidean distance:

$$d(x, x') = \sqrt{(x_1 - x_1')^2 + (x_2 - x_2')^2 + \cdots + (x_n - x_n')^2}$$

Where $d(x, x')$ is the distance between two instances x and x', and $x_1, x_2, \ldots, x_n$ are the features of the instances. The algorithm identifies the k-nearest neighbors to the test instance based on these distances. For classification tasks, the class label is determined by a majority vote among the k-nearest neighbors, while for regression tasks, the predicted value is calculated as the average (or weighted average) of the target values of these neighbors.

KNN is straightforward to implement and does not make strong assumptions about the underlying data distribution. However, its performance can be sensitive to the choice of k and the distance metric. Additionally, KNN can be computationally expensive for large datasets since it requires calculating distances to all training samples for each prediction.

3.1.3 Random Forest

Random Forest is an ensemble learning method that builds multiple decision trees and combines their outputs to make predictions. It employs bagging (Bootstrap Aggregating), where each tree is trained on a random bootstrap sample of the training data. A bootstrap sample is created by randomly selecting data points from the training set with replacement, meaning some points may be selected multiple times, while others may not be included. Additionally, at each split in the tree, a random subset of features is considered, introducing further randomness and reducing correlation among trees. The final prediction is determined by aggregating the outputs of all the trees, with the method of aggregation depending on whether the task is classification or regression.

1. For regression:
   - The final prediction is the average of the predictions from all individual trees:

     $$f(x) = \frac{1}{T} \sum_{t=1}^{T} f_t(x)$$

     where,
   - $f(x)$ is the final prediction for input x.
   - $f_t(x)$ is the prediction from the t-th decision tree.
   - T is the total number of trees in the forest.
2. For Classification:
   - In classification tasks, each decision tree in the Random Forest is trained to minimize the Gini impurity at each split. The Gini impurity for a node t is defined as:

     $$Gini(t) = 1 - \sum_{i=1}^{C} p_i^2$$

     Where p_i is the probability of an instance belonging to class i, and C is the number of classes.
   - The final prediction is made by taking the majority vote across all the trees, where each tree outputs a class label, and the class that receives the most votes is selected as the final prediction.

Random Forest is robust to overfitting and can handle high-dimensional data effectively. Its ensemble nature ensures better generalization compared to a single decision tree. However, it can be less interpretable due to the aggregation of multiple trees.

3.2 Application of Methods to the Data

We applied Logistic Regression, K-Nearest Neighbors (KNN), and Random Forest to predict bike demand. The features used included both quantitative and qualitative data:

- **Quantitative features**: Temperature (`temp`), humidity (`humidity`), wind speed (`windspeed`), precipitation (`precip`), snow depth (`snowdepth`), and visibility (`visibility`).
- **Qualitative features**: Hour of the day (`hour_of_day`), day of the week (`day_of_week`), holiday (`holiday`), and weekday (`weekday`).

3.2.1 Data Preprocessing

Before applying the models, we performed data preprocessing:

- **Feature scaling**: The numerical features were standardized using StandardScaler to ensure they were on a similar scale, which is especially important for distance-based models like KNN.
- **Encoding categorical variables**: The categorical features were encoded using one-hot encoding, converting them into binary columns.

3.2.2 Hyperparameter Tuning

For each model, we performed hyperparameter tuning to find the best configuration:

- For Logistic Regression, we applied grid search with k-fold cross-validation to identify the optimal combination of hyperparameters. This approach helped identify the optimal parameters while ensuring the model was well-generalized and avoided overfitting or underfitting. By testing multiple values for each hyperparameter, the best set of hyperparameters were chosen based on cross-validation accuracy.
- For KNN, we applied grid search with k-fold cross-validation to identify the best hyperparameters. We tested different values for k (from 1 to 100) to determine the optimal number of neighbors, and chosed the best value (K=12) using cross-validation. Cross-validation ensured that the selected hyperparameters minimized overfitting or underfitting. The best hyperparameters were chosen based on cross-validation accuracy, to ensure robust performance on unseen data.
- For Random Forest, we used GridSearchCV to find the optimal hyperparameters, including the number of trees (n-estimators) and the maximum depth (max-depth) of each tree. We tested different values for n-estimators (50, 100, 200) and max-depth (10, 20, 30), and choosed the optimal number of trees ($T = 100$) and the maximum depth of each tree ($d = 20$). Cross-validation was used to assess the model's performance on different subsets of the training data. The best combination of these parameters was selected based on cross-validation accuracy, resulting in a well-tuned model.

3.3 Model Evaluation

We evaluated the models using several performance metrics: - **Accuracy**: The proportion of correct predictions. - **Precision**: The proportion of true positive predictions among all positive predictions. - **Recall**: The proportion of true positives among all actual positives. - **F1-score**: The harmonic mean of precision and recall, useful for imbalanced datasets. The model evaluation of the classification methods after the hyperparameters tuning can be seen in the tabel below(Table 1) 1.

Table 1: Model Performance Comparison

Classification Methods	**Accuracy (High Bike Demand)**	**Precision (High Bike Demand)**	**Recall (High Bike Demand)**	**F1-Score (High Bike Demand)**
Logistic Regression	0.88	0.93	0.60	0.62
KNN	0.86	0.89	0.36	0.46
Random Forest	0.90	0.93	0.62	0.64

3.4 Model Selection

Based on the evaluation above shown in Tabel 1(1), we selected **Random Forest** to use "in production", as the final model for deployment. The key reasons for this decision are;

- **Higher performance**: Random Forest demonstrated the highest accuracy , Precision, Recall, and F1-score, than the other two models.
- **Robustness to overfitting**: Random Forest is less prone to overfitting due to averaging across multiple trees.
- **Handling both numerical and categorical data**: Random Forest can handle both types of features significant better, than KNN and logistic regression.

Thus, Random Forest was chosen as the model to be used "in production" to predict bike demand.

4 Conclusion

In this project, we developed a machine learning models to predict bike demand in Washington D.C.'s Capital Bikeshare system, focusing on determining when additional bikes should be made available. By analyzing a dataset containing various temporal and meteorological data features, we identified several key factors that influence bike demand, such as temperature, humidity, time of day, and whether it is a holiday or weekday.

We applied classification models such as, Logistic Regression, K-Nearest Neighbors (KNN), and Random Forest. After evaluating these models based on performance metrics such as accuracy, precision, recall, and F1-score, we found that **Random Forest** outperformed the other models in all categories. It achieved the highest accuracy of (0.90) and recall (0.62), making it the most reliable model for predicting bike demand in our case.

The Random Forest model was selected for deployment due to its superior performance, robustness to overfitting, and ability to handle both numerical and categorical features effectively.

Overall, this project demonstrates the use of machine learning in addressing urban transportation challenges.

References

1 Lindholm, A., et al. (2022) *Machine learning: A first course for engineers and scientists.* Cambridge: Cambridge University Press.

2 James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021) *An Introduction to Statistical Learning with Applications in R, Second Edition.* New York: Springer.

3 Wikipedia Contributors (2023). *K-nearest neighbors algorithm*. https://en.wikipedia.org/wiki/K-nearest_neighbors_algorithm

4 Wikipedia Contributors (2023). *Logistic regression*. https://en.wikipedia.org/wiki/Logistic_regression

5 Wikipedia Contributors (2023). *Random forest*. https://en.wikipedia.org/wiki/Random_forest

Appendix

In this appendix, we have Python code used to plot Figures for the project and to perform data analysis, model training, and evaluation for predicting bike demand in Washington D.C.

The following is the main code used in the report to plot figures.

```python
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
Created on Sun Dec 22 01:52:02 2024

@author: subashkc
"""

# Importing necessary libraries for data manipulation, plotting, and machine learning

import pandas as pd


import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
from scipy.stats import chi2_contingency

# Load the dataset
data = pd.read_csv("training_data_fall2024.csv")




# section 1 categorical data
# To PLot Figure 1 Categorical data vs Bike demand

# Set the style for the plots
sns.set(style="whitegrid")

# Create a figure with multiple subplots for each feature
fig, axes = plt.subplots(2, 3, figsize=(18, 12))  # 2 rows and 3 columns for subplots

# Count plot for Hour of Day
sns.countplot(x='hour_of_day', hue='increase_stock', data=data, palette='viridis', ax=axes[0, 0])
axes[0, 0].set_title('Bike Demand by Hour of Day')
axes[0, 0].set_xlabel('Hour of Day')
axes[0, 0].set_ylabel('Count')
axes[0, 0].legend(title='Increase Stock', loc='upper right', labels=['Low Demand', 'High Demand'])
axes[0, 0].tick_params(axis='x', rotation=45)

# Count plot for Day of Week
sns.countplot(x='day_of_week', hue='increase_stock', data=data, palette='viridis', ax=axes[0, 1])
axes[0, 1].set_title('Bike Demand by Day of Week')
axes[0, 1].set_xlabel('Day of Week')
axes[0, 1].set_ylabel('Count')
axes[0, 1].legend(title='Increase Stock', loc='upper right', labels=['Low Demand', 'High Demand'])
axes[0, 1].tick_params(axis='x', rotation=45)

# Count plot for Month
sns.countplot(x='month', hue='increase_stock', data=data, palette='viridis', ax=axes[0, 2])
axes[0, 2].set_title('Bike Demand by Month')
```

```python
axes[0, 2].set_xlabel('Month')
axes[0, 2].set_ylabel('Count')
axes[0, 2].legend(title='Increase Stock', loc='upper right', labels=['Low Demand', 'High Demand'])
axes[0, 2].tick_params(axis='x', rotation=45)

# Count plot for Holiday (Holiday vs. No Holiday)
sns.countplot(x='holiday', hue='increase_stock', data=data, palette='viridis', ax=axes[1, 0])
axes[1, 0].set_title('Bike Demand on  Non-Holiday(0) vs Holiday(1) ')
axes[1, 0].set_xlabel('Holiday')
axes[1, 0].set_ylabel('Count')
axes[1, 0].legend(title='Increase Stock', loc='upper right', labels=['Low Demand', 'High Demand'])

# Count plot for Weekday (Weekday vs. Weekend)
sns.countplot(x='weekday', hue='increase_stock', data=data, palette='viridis', ax=axes[1, 1])
axes[1, 1].set_title('Bike Demand on Weekend(0) vs Weekday(1)')
axes[1, 1].set_xlabel('Weekday')
axes[1, 1].set_ylabel('Count')
axes[1, 1].legend(title='Increase Stock', loc='upper right', labels=['Low Demand', 'High Demand'])

# Count plot for Summertime (Summertime vs Non-Summertime)
sns.countplot(x='summertime', hue='increase_stock', data=data, palette='viridis', ax=axes[1, 2])
axes[1, 2].set_title('Bike Demand in  Non-Summertime(0) vs Summertime(1) ')
axes[1, 2].set_xlabel('Summertime')
axes[1, 2].set_ylabel('Count')
axes[1, 2].legend(title='Increase Stock', loc='upper right', labels=['Low Demand', 'High Demand'])

# Adjust layout to avoid overlap
plt.tight_layout()
#plt.savefig('categorical features.png')
plt.show()









# Numerical feature plot part

# Encode target variable ('increase_stock') to binary values
data['increase_stock'] = data['increase_stock'].map({'low_bike_demand': 0, 'high_bike_demand': 1})

# Define the numerical features to analyze
numerical_features = ['temp', 'dew', 'humidity', 'windspeed', 'precip', 'snowdepth', 'visibility', 'cloudcover']

# Function to divide numerical feature into bins and calculate proportion of high demand
def calculate_proportion_of_high_demand(df, feature, target_feature, bins):
    # Divide the numerical feature into intervals (bins)
    df['binned'] = pd.cut(df[feature], bins=bins, right=False)  # right=False to make intervals left-inclusive
    # Group by the binned intervals and calculate the proportion of high demand
```

```
    high_demand_proportion = df.groupby('binned')[target_feature].mean()  # Mean of target will give proportion of 1 (high demand)
    return high_demand_proportion

# Function to perform Chi-square test for each feature's binned intervals
def chi_square_test_for_binned_data(df, feature, target_feature, bins):
    # Calculate observed frequencies for each bin
    df['binned'] = pd.cut(df[feature], bins=bins, right=False)
    contingency_table = pd.crosstab(df['binned'], df[target_feature])
    # Perform Chi-square test
    chi2, p, _, _ = chi2_contingency(contingency_table)
    return chi2, p, contingency_table

# Define the intervals (bins) for each numerical feature (e.g., temperature)
temp_bins = [0, 10, 20, 30, 40]  # Example bins for temperature (0-10, 10-20, 20-30, 30-40)
humidity_bins = [0, 30, 60, 90]  # Example bins for humidity
windspeed_bins = [0, 5, 10, 20]  # Example bins for windspeed
precip_bins = [0, 1, 5, 10]  # Example bins for precipitation
snowdepth_bins = [0, 5, 10, 20]  # Example bins for snow depth
visibility_bins = [0, 5, 10, 20]  # Example bins for visibility
cloudcover_bins = [0, 25, 50, 75, 100]  # Example bins for cloud cover

# List of bins for each feature
bins_dict = {
    'temp': temp_bins,
    'dew': temp_bins,  # Using the same bins for dew point as temperature and so on
    'humidity': humidity_bins,
    'windspeed': windspeed_bins,
    'precip': precip_bins,
    'snowdepth': snowdepth_bins,
    'visibility': visibility_bins,
    'cloudcover': cloudcover_bins
}

# Dictionary to store the results for all numerical features
proportions_dict = {}
chi_square_results = {}

# Loop through each numerical feature and perform the analysis
for feature in numerical_features:
    # Calculate proportion of high demand for the feature
    proportions_dict[feature] = calculate_proportion_of_high_demand(data, feature, 'increase_stock', bins_dict[feature])

    # Perform Chi-square test for the feature
    chi2_stat, p_value, contingency_table = chi_square_test_for_binned_data(data, feature, 'increase_stock', bins_dict[feature])
    chi_square_results[feature] = {
        'chi2_stat': chi2_stat,
        'p_value': p_value,
        'contingency_table': contingency_table
    }

# Create a grid of subplots for all numerical features (2 rows, 4 columns)
fig, axes = plt.subplots(2, 4, figsize=(20, 10))  # 2 rows, 4 columns

# Flatten the axes to easily loop over them
axes = axes.flatten()
```

```python

# Feature descriptions
feature_descriptions = {
    'temp': 'Temperature in Celsius degrees.',
    'dew': 'Dew point in Celsius degrees.',
    'humidity': 'Relative humidity (percentage)',
    'windspeed': 'Wind speed in km/h',
    'precip': 'Precipitation in mm',
    'snowdepth': 'Amount of snow in the last hour in mm',
    'visibility': 'Distance in km at which objects or landmarks can be clearly seen and identified',
    'cloudcover': 'Percentage of the city covered in clouds'
}

# Plot the proportions of high demand for each feature
for i, feature in enumerate(numerical_features):
    # Plot the proportions
    axes[i].bar(proportions_dict[feature].index.astype(str), proportions_dict[feature], color='skyblue')

    # Set title and labels
    axes[i].set_title(f"Proportion of High Bike Demand by {feature.capitalize()}")
    axes[i].set_xlabel(f"{feature.capitalize()} Range")
    axes[i].set_ylabel("Proportion of High Demand")
    axes[i].tick_params(axis='x', rotation=45)

    # Add feature description in the title
    axes[i].text(0.5, 0.95, feature_descriptions[feature], ha='center', va='top', fontsize=10, color='black', transform=axes[i].transAxes)

    # Set y-axis to display the feature's range for context
    axes[i].set_ylim(0, 1)  # Keep y-axis consistent for all plots (proportion ranges from 0 to 1)

    # Optionally, set the x-ticks as ranges based on the bins
    if feature in bins_dict:
        bin_labels = [f"{bins_dict[feature][i]}-{bins_dict[feature][i+1]}" for i in range(len(bins_dict[feature])-1)]
        axes[i].set_xticklabels(bin_labels, rotation=45, ha='right')

# Adjust layout for better spacing
plt.tight_layout()
#plt.savefig('Numerical features.png')
plt.show()

# Print Proportions and Chi-square Results for all features
#for feature in numerical_features:
#    print(f"Proportion of High Demand for {feature}:")
#    print(proportions_dict[feature])
#    print(f"\nChi-square Test Results for {feature}:")
#    print(f"Chi2 Statistic: {chi_square_results[feature]['chi2_stat']:.4f}, p-value: {chi_square_results[feature]['p_value']:.4f}")
#   print(f"Contingency Table:\n{chi_square_results[feature]['contingency_table']}\n")
```

Listing 1: Python Code for plotting figures

The following is the main code used in this project for the machine learning parts :

```python
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
Created on Sun Dec 22 01:52:02 2024

```

```python
@author: subashkc
"""

# Import necessary libraries
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score, classification_report

# Load the dataset
data = pd.read_csv("training_data_fall2024.csv")

# Encode the target variable
data['increase_stock'] = data['increase_stock'].map({'low_bike_demand': 0, 'high_bike_demand': 1})

# Define numerical and categorical features
numerical_features = ['temp', 'dew', 'humidity', 'windspeed', 'precip', 'snowdepth', 'visibility', 'cloudcover']
categorical_features = ['hour_of_day', 'day_of_week', 'month', 'holiday', 'weekday', 'summertime']

# Prepare features (X) and target (y)
X = data[numerical_features + categorical_features]
y = data['increase_stock']

# Define preprocessor for numerical and categorical features
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numerical_features),
        ('cat', OneHotEncoder(), categorical_features)
    ]
)

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

##############################
# 1. Logistic Regression
##############################

# Define Logistic Regression model
log_reg = LogisticRegression(max_iter=1000, random_state=42)

# Create a pipeline for Logistic Regression
pipeline_log_reg = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', log_reg)
])

# Define hyperparameter grid for Logistic Regression
param_grid_log_reg = {

}

# Perform Grid Search with Cross-Validation for Logistic Regression
```

```python
grid_search_log_reg = GridSearchCV(pipeline_log_reg, param_grid_log_reg, cv=5, scoring='accuracy', n_jobs=-1)
grid_search_log_reg.fit(X_train, y_train)

# Get the best model and parameters
best_model_log_reg = grid_search_log_reg.best_estimator_
best_params_log_reg = grid_search_log_reg.best_params_

# Print best hyperparameters
print("Logistic Regression Best Hyperparameters:", best_params_log_reg)

# Evaluate on the test data
y_pred_log_reg = best_model_log_reg.predict(X_test)
print("Logistic Regression Accuracy:", accuracy_score(y_test, y_pred_log_reg))
print(classification_report(y_test, y_pred_log_reg))



##############################
# 2. K-Nearest Neighbors (KNN)
##############################

# Define KNN model
knn = KNeighborsClassifier()

# Create a pipeline for KNN
pipeline_knn = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', knn)
])


# Define a  range of k values,
k_values = list(range(1, 100))  # Testing k from 1 to 100


# Define the hyperparameter grid for KNN
param_grid_knn = {
    'classifier__n_neighbors': k_values ,  # Number of neighbors
    'classifier__weights': ['distance'],  # Weight options
    'classifier__metric': ['euclidean'],  # Distance metrics
    'classifier__p': [ 2]  # p=1 for Manhattan, p=2 for Euclidean
}

# Perform Grid Search with Cross-Validation for KNN
grid_search_knn = GridSearchCV(pipeline_knn, param_grid_knn, cv=5, scoring='accuracy', n_jobs=-1)
grid_search_knn.fit(X_train, y_train)

# Get the best model and parameters
best_model_knn = grid_search_knn.best_estimator_
best_params_knn = grid_search_knn.best_params_

# Print best hyperparameters
print("KNN Best Hyperparameters:", best_params_knn)

# Evaluate on the test data
y_pred_knn = best_model_knn.predict(X_test)
print("KNN Accuracy:", accuracy_score(y_test, y_pred_knn))
print(classification_report(y_test, y_pred_knn))
```

```python
##############################
# 3. Random Forest Classifier
##############################

# Random Forest Classifier: An ensemble model that builds multiple decision trees
rf = RandomForestClassifier(n_estimators=100, random_state=42)  # Initialize the model with 100 trees
rf.fit(X_train, y_train)  # Fit the model on the training data

# Random Forest Predictions and Evaluation
y_pred_rf = rf.predict(X_test)  # Predict on the test data
print("Random Forest Accuracy:", accuracy_score(y_test, y_pred_rf))  # Accuracy of the model
print(classification_report(y_test, y_pred_rf))  # Detailed classification report

# GridSearch for Random Forest hyperparameter tuning: Finding the best combination of parameters
param_grid = {
    'n_estimators': [50, 100, 200],  # Number of trees in the forest
    'max_depth': [10, 20, 30],  # Maximum depth of the trees
}
grid_search = GridSearchCV(estimator=RandomForestClassifier(random_state=42), param_grid=param_grid, cv=3)  # 3-fold cross-validation
grid_search.fit(X_train, y_train)  # Perform grid search on the training data
print("Best Parameters from GridSearch:", grid_search.best_params_)  # Print the best parameters found

# Evaluate the best Random Forest model from GridSearch
best_rf = grid_search.best_estimator_  # Get the best model
y_pred_best_rf = best_rf.predict(X_test)  # Predict on the test data
print("Best Random Forest Report:\n", classification_report(y_test, y_pred_best_rf))  # Classification report for the best model
```

Listing 2: Python Code for plotting figures